Secure By Design

EDUCATE. EMPOWER. SECURE THE FUTURE.

COURSE 1.12 / COMPANION WORKBOOK

Analyze the record in SBD-SPLUNK01

Search a working copy, own the alerts, prove the chain

The last of three logging courses: build SBD-SPLUNK01 as a domain-joined Linux server patched only from SBD-REPO01, install Splunk Enterprise from hashed media, forward the collector's two streams over TLS, build the searches that answer this network's first questions, turn them and the pipeline's own failures into alerts someone owns, and trace one event from its source through the collector and the repository to the analyst's screen.

MATERIALS EDITION 1.0 / COURSE 1.12

Updated 14 October 2026 · sbd-splunk-analysis

NAME _____ / DATE _____

© 2026 Secure By Design Team LLC.

START HERE / YOUR LEARNING PATH

Table of contents

Read each lesson alongside the interactive guide and the video. Use the record tables to write down what you actually observe — the workbook is your evidence, not the guide.

Before you start

01 Plan analysis and build SBD-SPLUNK01

02 Install Splunk and bring in the streams

03 Build the working searches

04 Own the alerts

05 Prove the chain and hand off

Course review and sign-off

> **HOW TO USE IT** Each lesson page has the same shape: what you will learn, the build steps, the commands that verify them, and a record table. Fill the table from your own environment — never from the sample values.

Before you start

This is the last of three logging courses: course 1.10 collects the record on SBD-SYSLOG01, course 1.11 protects it on SBD-AUDIT01, and this course analyzes a working copy in SBD-SPLUNK01, then traces one event through all three. Analysis is a search someone actually runs and an alert someone owns. Linux commands are for the Red Hat Enterprise Linux–family release course 1.9 chose. Retention periods, alert thresholds and alert expirations depend on your environment and policy: the course says what each one is for, and you record the value you chose.

Prerequisites

- ☐ Complete courses 1.10 and 1.11: SBD-SYSLOG01 collects the two streams — the ForwardedEvents log on D:\Logs\ForwardedEvents, sized for the buffer course 1.10 recorded, and daily syslog files per sender under D:\Logs\Syslog — SBD-AUDIT01 archives them every night, and the register carries row F10 waiting for this course.
- ☐ Provision a virtual machine for SBD-SPLUNK01 on VLAN 50 (10.0.50.10) with a separate data volume for the indexes (/srv/splunk in this sample), and have course 1.9's client package for SBD-REPO01 and its Linux assessment set (SCC for Linux and the distribution's STIG benchmark in C:\Staging\DISA\linux on SBD-ADMIN-SRV01), course 1.3's chrony file, course 1.6's root certificate hash and SBD Server Authentication template, and course 1.8's administration seats ready.
- ☐ Stage across the air gap, hashed as in course 1.4: the Splunk Enterprise package for Linux (.tgz), the universal forwarder installer for Windows (.msi), the Splunk Add-on for Microsoft Windows, an Enterprise license file, and DISA's Splunk Enterprise STIG for lesson 5's checklist (the Linux scan reuses course 1.9's set).

What this course builds

- Plan analysis — indexes, a retention period set for your environment, roles, and console paths — and build SBD-SPLUNK01 on domain time, patched only from SBD-REPO01.
- Install Splunk Enterprise and forward the collector's two streams over TLS, with counts and times that match the collector.
- Build the working searches and turn them, and the pipeline's own failures, into tested alerts with named responders.
- Trace one event from its source through the collector and the repository to the analyst's screen.

Systems in scope

RECORD	YOUR OBSERVATION
SBD-SPLUNK01 — name, address, OS build	
SBD-SYSLOG01 — name, address, OS build	

LESSON 01 / ABOUT 35 MIN

Plan analysis and build SBD-SPLUNK01

> **WHY THIS MATTERS** Analysis works on a copy, not on the record: SBD-AUDIT01 keeps the evidence, and SBD-SPLUNK01 keeps a searchable working copy, for a period your environment sets, for the people who question them. Decide the indexes, retention, roles, and console paths first, then build the domain-joined Linux host that runs it — patched only from SBD-REPO01 and on domain time from its first boot.

In this lesson you will

- Define what analysis is for, its indexes and the retention you choose, and who may do what from where.
- Stage the Splunk media across the air gap with verified hashes.
- Build SBD-SPLUNK01 as a domain-joined Linux server on domain time, patched only from SBD-REPO01.

Follow the build

1. Write analysis's job and its limits: SBD-SPLUNK01 indexes a working copy of the collector's two streams (rule F10) for searches, dashboards, and alerts, while the protected record stays on SBD-AUDIT01 — when an investigation needs evidence, it retrieves the archive. Plan one index per stream, sbd_wineventlog for the forwarded Windows events and sbd_syslog for the syslog files, each searchable for as long as investigations in your environment need to look back and then deleted, because the repository already keeps the record for its own retention period. The period depends on your environment and policy; record the value you chose and why. Size each from a typical day times that period plus headroom, starting from course 1.11's daily figures until lesson 2 measures what Splunk actually stores, and cap its size above that, so age rather than size retires the data.
2. Decide who does what: analysts (sbd_analyst) search and review alerts but change nothing, administrators (sbd_admin) manage inputs, indexes, roles, and alerts, and no role used by people gets delete_by_keyword, so nothing in the working copy can be quietly removed. Everyone signs in with a named Splunk account, so no domain password is typed into the console, and the admin account made at install is sealed like course 1.8's break-glass account. Splunk Web and SSH answer VLAN 60 only: analysts and the host's Tier 1 administrators work from SBD-ADMIN-SRV01 (rule A02), and the Tier 0 operators who own the alerts on the domain controllers, the model's DC-facing scope, from AdminJumpBox01 (rule A01). Splunk's roles can't fence one administrator's alerts from another's, and root on the host can change any Splunk file, so keep both groups small and every change on record.
3. Stage the media as course 1.4 taught: the Splunk Enterprise package for Linux (.tgz) and the universal forwarder installer for Windows (.msi), each checked on the connected side against the SHA-512 that Splunk publishes for it, plus the Splunk Add-on for Microsoft Windows, your Enterprise license file, and DISA's Splunk Enterprise STIG. Hash the set for the transfer record and check it again on the lab side before anything is opened. Don't plan on the 60-day trial license: Splunk Free has no users, roles, or alerts, and an Enterprise license installs from its file with no connection to Splunk.
4. Build SBD-SPLUNK01 from hashed install media of the Red Hat Enterprise Linux–family release course 1.9 chose, with the VM's hypervisor time synchronization turned off so the domain controllers are its only clock: the name SBD-SPLUNK01.sbd.test, the static address 10.0.50.10/24 with the core's VLAN 50 interface as gateway (10.0.50.1 in this sample), both domain controllers as DNS servers, and the data volume mounted at /srv/splunk. Before its first update, apply course 1.9's client package, checked

against its hashes, so SBD-REPO01 is its only package source (rule F19) and no other source remains; then update from the current snapshot.

5. Install course 1.3's chrony file, both domain controllers with maxdistance 16, and confirm a controller is selected with a small, steady offset before anything else: Kerberos refuses a clock that is too far from the domain's. Then join sbd.test with realm join --computer-ou='OU=Servers,OU=SBD,DC=sbd,DC=test' and a Tier 1 account, which course 1.7 allowed to create computers there (rules F01, F02, and F03), permit sign-in only to SBD Tier 1 Admins with realm permit -g, and give that group sudo through a drop-in file; SSSD then registers the host's address in DNS.
6. Make the host firewall the register's copy, with firewalld: SSH from 10.0.60.20, Splunk Web on TCP 8000 from 10.0.60.10 and 10.0.60.20, and TCP 9997 from SBD-SYSLOG01 (10.0.20.20) alone. Then send the host's own syslog, sudo use included, to the collector over TLS (rule F07) as course 1.10 enrolled SBD-REPO01: rsyslog-gnutls from the snapshot, the course 1.6 root placed in the system trust store by hand as course 1.9 placed it on SBD-REPO01 (exported on SBD-ADMIN-SRV01 and its SHA-256 checked against course 1.6's record), the collector's certificate name checked, and SBD-SPLUNK01 added to the collector's TCP 6514 sender list.

Check your result

```
# On SBD-SPLUNK01 (Tier 1 SSH session from SBD-ADMIN-SRV01)
cd /srv/media && sha256sum -c --quiet SHA256SUMS-splunk.txt   # expected: no output
dnf repolist --all              # expected: only the sbd-repo01 repositories enabled
chronyc sources -v; chronyc tracking   # ^* one controller, ^+ or ^- the other
realm list                      # sbd.test, kerberos-member, one permitted group
openssl x509 -inform der -in /tmp/sbd-root.cer -noout -fingerprint -sha256   # matches
course 1.6's record
firewall-cmd --get-active-zones
logger -t sbd-splunk-proof 'SBD-SPLUNK01 syslog over TLS test'
# On SBD-SYSLOG01 (Tier 1 session): the test message in SBD-SPLUNK01's daily file
Select-String -Path D:\Logs\Syslog\*\*.log -Pattern 'sbd-splunk-proof' | Select-Object -
First 1
```

> **WHAT SUCCESS LOOKS LIKE** The design names analysis's job, both indexes with their retention and sizing, the roles and who holds them, and the console paths; the media matches its published and transfer hashes; and SBD-SPLUNK01 resolves to 10.0.50.10, lists SBD-REPO01 as its only package source, has a controller selected with a small, stable offset, is joined with one permitted group, trusts the course 1.6 root whose hash matches that course's record, admits only the registered paths, and its test message arrives on the collector.

Your record

Evidence to keep: Keep the analysis design (job, indexes, the retention period you chose and why, sizing, roles, console paths), the media hashes with the transfer record, and SBD-SPLUNK01's build record: package sources, chrony output, realm list, the root certificate's hash compared with course 1.6's record, firewall zones, and its first syslog line on the collector.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	

RECORD	YOUR OBSERVATION
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

What is SBD-SPLUNK01's copy of the record for?

> **REMEMBER** Analysis keeps a working copy for the period you chose; the record stays on SBD-AUDIT01. Named accounts in two roles, no delete, consoles on VLAN 60 only. SBD-SPLUNK01 is joined, on domain time, and patched only from SBD-REPO01.

Splunk · product documentation

Splunk · Set a retirement and archiving policy

Splunk · Define roles on the Splunk platform with capabilities

Red Hat · Integrating RHEL systems directly with Windows Active Directory

NIST · SP 800-53 Rev. 5 security controls (AU family)

LESSON 02 / ABOUT 40 MIN

Install Splunk and bring in the streams

WHY THIS MATTERS Install Splunk Enterprise on SBD-SPLUNK01 with the indexes, roles, and TLS receiving port lesson 1 designed, then put a universal forwarder on SBD-SYSLOG01 that reads the collector's two streams and sends them over rule F10. Finish by proving the copy is faithful: Splunk's counts and times must match the collector's.

In this lesson you will

- Install Splunk Enterprise with HTTPS, the designed indexes and roles, and a TLS receiving port.
- Forward both of the collector's streams from SBD-SYSLOG01 over rule F10 with TLS.
- Show that Splunk's event counts and timestamps match the collector's.

Follow the build

1. As root on SBD-SPLUNK01, create the splunk account (the package doesn't), extract the package with tar -xzf into /opt, and give splunk ownership of /opt/splunk and /srv/splunk. Before the first start, add SPLUNK_DB=/srv/splunk to /opt/splunk/etc/splunk-launch.conf and, where your rules require FIPS mode, SPLUNK_FIPS=1: Splunk can enter FIPS mode only at its first start (it turns it on by itself if the Linux kernel already runs in FIPS mode), and an instance started without it can't gain it later, even by upgrading. Start it as splunk with --accept-license, create the admin account when prompted and seal its password, install the license file, and enable boot-start under systemd for the splunk user.
2. Request one SBD Server Authentication certificate for SBD-SPLUNK01.sbd.test through the course 1.6 process, with the key and request made on the host with openssl and the name in both subject and subject alternative name, and keep the key readable by splunk alone. Turn on HTTPS for Splunk Web in web.conf (enableSplunkWebSSL, serverCert, privKeyPath), install the Splunk Add-on for Microsoft Windows, and, in a small app of your own, create both indexes with frozenTimePeriodInSecs set to lesson 1's retention period in seconds and the sized maxTotalDataSizeMB, and the two roles: sbd_analyst inheriting the built-in user role, sbd_admin inheriting admin. Create each person's named account with one role, then sign out of the admin account.
3. Open the receiving port in inputs.conf: a [splunktcp-ssl:9997] stanza and an [SSL] stanza whose serverCert is one PEM file holding the certificate, its key, and the SBD chain, with sslVersions = tls1.2. The forwarder checks the server's certificate, and requireClientCert stays false: as on the collector's syslog path in course 1.10, the sender is held to its address, by the host firewall's rule that admits 10.0.20.20 alone.
4. On SBD-SYSLOG01, in a Tier 1 session, install the forwarder with msiexec (AGREETOLICENSE=Yes LAUNCHSPLUNK=0 /quiet); by default it runs as a low-privilege virtual account, NT SERVICE\SplunkForwarder. The STIG's user-rights policy replaces any rights the installer grants, so give the account what it needs explicitly: Event Log Readers membership, added beside SBD-Archive$ in the course 1.11 GPO as a second add-member preference item so neither account replaces the other, and read on D:\Logs\Syslog, both recorded in the course 1.5 exception record. Copy the Windows add-on into the forwarder's apps folder and enable two inputs in its local inputs.conf: [WinEventLog://ForwardedEvents] with renderXml = true and index = sbd_wineventlog, and [monitor://D:\Logs\Syslog] with sourcetype = syslog, index = sbd_syslog, and host_segment = 3, so each line's host is its sender, not the collector.

5. Point it at rule F10 in outputs.conf: server = SBD-SPLUNK01.sbd.test:9997, useSSL = true, sslVerifyServerCert = true, sslAltNameToCheck = SBD-SPLUNK01.sbd.test, and useACK = true, so the forwarder keeps each block until the indexer confirms it. In server.conf, set sslRootCAPath to the SBD chain and turn off the forwarder's own management port ([httpServer] disableDefaultPort = true): nothing manages it remotely, and course 1.10 allows no listener the design didn't ask for. Start the service; on its first run the forwarder reads ForwardedEvents from the oldest event, so the whole buffer arrives, and its size on disk, divided by the days it covers, replaces lesson 1's sizing estimate.
6. Prove the copy is faithful. Pick one closed hour, count it on the collector with Get-WinEvent and in Splunk over the same window, and compare one event on both sides: the counts must match, and the event's time in Splunk must be when its source wrote it, not when it arrived. Do the same for yesterday's syslog file from each sender, check that host names each source system rather than SBD-SYSLOG01, and confirm in the indexer's own metrics that the forwarder's connection is TLS (connectionType cookedSSL).

Check your result

```
# On SBD-SPLUNK01 (Tier 1 SSH session): the first-start decisions, an index, and the TLS
port
grep -E '^SPLUNK_(DB|FIPS)' /opt/splunk/etc/splunk-launch.conf
sudo -u splunk /opt/splunk/bin/splunk btool indexes list sbd_wineventlog | grep -E
'frozenTimePeriodInSecs|maxTotalDataSizeMB'
sudo -u splunk /opt/splunk/bin/splunk btool inputs list splunktcp-ssl:9997 --debug
openssl s_client -connect SBD-SPLUNK01.sbd.test:9997 -CAfile /opt/splunk/etc/auth/sbd/sbd-
chain.pem </dev/null 2>/dev/null | grep -E 'Verify return code|Protocol'
# On SBD-SYSLOG01 (Tier 1 session): the forwarder's account and connection, and one closed
hour
Get-CimInstance Win32_Service -Filter "Name='SplunkForwarder'" | Select-Object StartName,
State
Select-String 'C:\Program Files\SplunkUniversalForwarder\var\log\splunk\splunkd.log' -
Pattern 'Connected to idx' | Select-Object -Last 1
$start = (Get-Date).Date.AddHours((Get-Date).Hour - 2); $end = $start.AddHours(1)
(Get-WinEvent -FilterHashtable @{LogName='ForwardedEvents'; StartTime=$start;
EndTime=$end}).Count
# In Splunk Web (Search), as sbd_admin: the same hour, the forwarder's connection, and the
stored size
index=sbd_wineventlog earliest=-2h@h latest=-1h@h | stats count
index=_internal source=*metrics.log group=tcpin_connections | stats latest(connectionType)
AS type, latest(version) AS version BY hostname, sourceIp
| dbinspect index=sbd_wineventlog index=sbd_syslog | stats sum(sizeOnDiskMB) AS MB BY
index
```

> **WHAT SUCCESS LOOKS LIKE** Splunk runs as splunk under systemd with the designed indexes and retention, Splunk Web answers over HTTPS, TCP 9997 presents the SBD-SPLUNK01 certificate over TLS 1.2, the forwarder runs as NT SERVICE\SplunkForwarder with a TLS connection to the indexer, and a closed hour's count and one event's time match the collector for both streams.

Your record

Evidence to keep: Keep the first-start settings and the FIPS decision, the btool listings for the indexes and the receiving port, the TLS handshake result, the forwarder's inputs and outputs files and its service account, the measured daily size, and the count-and-time comparison for both streams.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Why do SPLUNK_DB and the FIPS decision go into splunk-launch.conf before the first start?

> **REMEMBER** Splunk runs as its own account, with HTTPS and indexes that keep the period you chose. Both streams cross rule F10 over TLS, checked by name. Counts and times in Splunk match the collector's.

Splunk · product documentation

Splunk · Secure Splunk Enterprise with FIPS

Splunk · Configure Splunk indexing and forwarding to use TLS certificates

Splunk · Configure Splunk Web to use TLS certificates

Splunk · Set a retirement and archiving policy

Splunk · Configure the Splunk Add-on for Windows (forwarded events)

Microsoft · Get-WinEvent

LESSON 03 / ABOUT 30 MIN

Build the working searches

> **WHY THIS MATTERS** Analysis earns its place with three working searches: failed privileged sign-ins, the boundary's denies, and the health of time — questions this network can already answer about itself. Build them as an analyst, prove each against an event you caused, and save them for the alerts in lesson 4.

In this lesson you will

- Search as an analyst and read the shape of the indexed data.
- Build the first three searches against real events.
- Save each with its owner and a known test event.

Follow the build

1. Sign in from SBD-ADMIN-SRV01 with your named sbd_analyst account and check what the role allows: | rest /services/authentication/current-context lists your roles and capabilities, and delete_by_keyword is not among them. Then read the shape of the data with | tstats count where index=sbd_* by index, host, sourcetype: every source system should appear, the Windows events with their source computer in host and their event ID in EventCode, and the syslog lines with their sender in host.
2. Build search one: failed sign-ins (event 4625) involving the tier admin accounts, grouped by source host — the tiering course's denials now have a dashboard. Take the accounts from course 1.8's tier groups (SBD Tier 0 Admins, SBD Tier 1 Admins, and SBD Tier 2 Admins), not from a naming pattern: from AdminJumpBox01, a Tier 0 operator exports the groups' members to sbd_tier_admins.csv (TargetUserName and tier), uploads it as a lookup the analysts can read, and refreshes it after every membership change.
3. Build search two: router denies by source VLAN and destination port — the network boundary, now trending. Extract src_ip, dest_ip, and dest_port from the core's deny lines (its deny log reaches the collector as syslog on UDP 514 under rule F24, part of the network's existing configuration) once with the field extractor, and derive each VLAN from an address's third octet, since every segment is 10.0.<VLAN>.0/24.
4. Build search three: time-service events, so clock drift surfaces before Kerberos complains. Course 1.10's SBD-ForwardedSecurity already carries Security event 4616 and the System log's Microsoft-Windows-Time-Service events course 1.3 keeps (12, 35, 36, 37, 129, and 142); event 12 on SBD-DC01 is expected, because the PDC emulator serves its own clock. The Linux servers report through chronyd in the syslog stream: which source they selected, or that none was selectable.
5. Save each as a report with its question, owner, the events it depends on, and a known event it must find, and have an sbd_admin share it with the analysts read-only: a wrong password for your Tier 1 account at SBD-ADMIN-SRV01's sign-in screen for search one, a probe of an unregistered port for search two, and a chronyd restart on SBD-SPLUNK01 for search three. Cause each, run each search, and keep the result: lesson 4 turns them into alerts.

Check your result

```
# In Splunk Web (Search), signed in as an analyst from SBD-ADMIN-SRV01
| rest /services/authentication/current-context splunk_server=local | table username roles
capabilities
| tstats count where index=sbd_* by index, host, sourcetype
```

```
index=sbd_wineventlog EventCode=4625 [| inputlookup sbd_tier_admins.csv | fields
TargetUserName] | stats count, values(IpAddress) AS sources, latest(_time) AS last BY
host, TargetUserName | convert ctime(last)
index=sbd_syslog host=<core> src_ip=* | eval src_vlan="VLAN ".mvindex(split(src_ip, "."),
2) | stats count BY src_vlan, dest_port
(index=sbd_wineventlog "Microsoft-Windows-Time-Service") OR (index=sbd_syslog chronyd) |
fillnull value="-" EventCode | stats count, latest(_time) AS last BY host, EventCode |
convert ctime(last)
# On SBD-SYSLOG01 (Tier 1 session): course 1.10's subscription carries the Time-Service
query
wecutil gs SBD-ForwardedSecurity /f:xml | Select-String 'Time-Service'
```

WHAT SUCCESS LOOKS LIKE The analyst role searches both indexes and carries no delete capability, every source system appears with the right host and source type, the subscription carries the Time-Service query, and each saved search finds its known test event.

Your record

Evidence to keep: Record your role listing, the data-shape table, each saved search with its question, owner, and a result showing its test event, the lookup's source and date, and the subscription's Time-Service query.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Why does the repository stay out of the analysis flow?

REMEMBER Analysts question the working copy; the protected record stays untouched. Three searches answer real questions from day one. Each search has an owner and a test event it must find.

Splunk · Define roles on the Splunk platform with capabilities

Splunk · Configure the Splunk Add-on for Windows (forwarded events)

Microsoft · Use Windows Event Forwarding to help with intrusion detection

Microsoft · Wecutil (Windows Event Collector)

NIST · SP 800-53 Rev. 5 security controls (AU family)

LESSON 04 / ABOUT 35 MIN

Own the alerts

> **WHY THIS MATTERS** An alert nobody owns is a log entry with extra steps. Turn the working searches and the pipeline's own failure conditions into scheduled alerts, each with a threshold set for your environment, a named responder, and a written first response; test each one by causing it, tune the noise, and set the review cadence AU-6 asks for.

In this lesson you will

- Write each alert's threshold, schedule, named responder, and first response, recording the values you chose.
- Alert on the pipeline's own failures and the repository's findings.
- Test and tune every alert, and review them on a set cadence.

Follow the build

1. Write the alert register before building anything: for each alert, its search, threshold, schedule, severity, named responder with a deputy, and written first response. The threshold, schedule, throttle and expiration of each alert depend on your environment and policy, so record the value you chose and why. Start from lesson 3's searches: any failed sign-in for a tier account, since privileged sign-ins are few and each failure deserves a look; any deny from VLAN 80 toward VLAN 60 or from VLAN 60 itself, whose seats use only registered paths, or a burst of denies from one source above the count and window you record; and Time-Service events 36, 129, or 142, or chronyd finding no selectable source (never event 12, which SBD-DC01 writes by design). Add any use of a sealed account: course 1.8's break-glass account signing in anywhere, as that course promised, and Splunk's own admin account signing in to Splunk.
2. Add the conditions course 1.10's daily check has watched by hand (AU-5): no Windows events at all for longer than the silence threshold you record, which means the collector, its forwarder, or rule F10 has stopped, and a known source quiet past its own limit, taken from a lookup of expected sources (each host as Splunk shows it, with the longest quiet it may have). Add course 1.11's repository findings: SBD-Repository events 1001 and 1101 from SBD-AUDIT01, refused access to its archive (event 4656 with an audit failure), and any deletion (event 4660), which the responder matches to an approved retention run. Splunk sees missing data, not disks: the collector's free space and its buffer's oldest event stay in the daily check.
3. Build each as a scheduled alert owned by sbd_admin, the ones on the domain controllers from AdminJumpBox01: a cron schedule and a search window by index time, the window longer than the interval between runs (earliest=-24h _index_earliest=-<window>), so events that arrive late after an outage are still checked, a trigger when the number of results is greater than zero (so a missing-data search, such as the silence alert, is written to return a row only when its count is zero), throttling on the field that names one problem (the account, the source) for the period you record, a severity, and the Add to Triggered Alerts action. This network has no mail server, so the Triggered Alerts list is the responder's queue, and it keeps each entry for 24 hours by default: set each alert's expiration to cover the longest gap between reviews in your environment, so a weekend can't empty it, and record the value. Name every alert SBD - <condition>, so the register, the queue, and the scheduler's log use the same names.
4. Test each alert by causing its condition where that is safe, and record when you caused it, when it fired, and who saw it: one wrong password for your Tier 1 account at SBD-ADMIN-SRV01's sign-in screen, a

probe of an unregistered port from SBD-ADMIN-SRV01 across the core, and the SplunkForwarder service stopped on SBD-SYSLOG01 for longer than the silence threshold, which also shows the backlog arriving without a gap once it restarts. Where a condition can't be caused safely, such as a repository mismatch or a break-glass sign-in, run the alert's search over a window that holds a real or test example instead.

5. Tune before you trust: run each alert's search over the last seven days and count how often it would have fired. If that is more than the responder can act on, raise the threshold, widen the throttle, or exclude a known benign source, never just lower the severity, and record each change and its reason in the register.
6. Set the review cadence (AU-6; sample values): the responder works the queue at the start of each day and records every alert's outcome; once a week an analyst reviews each saved search's results and signs the review; and each month the register's thresholds, noise, and owners are reviewed, with both lookups refreshed after any change to the tier groups or the sources.

Check your result

```
# In Splunk Web (Search), as sbd_admin: the alerts as built
| rest /servicesNS/-/-/saved/searches splunk_server=local | search title="SBD - *"
is_scheduled=1 | table title cron_schedule dispatch.earliest_time alert.severity
alert.suppress.period alert.expires actions
# Each alert's runs and what it triggered, from the scheduler's own log
index=_internal sourcetype=scheduler savedsearch_name="SBD - *" | stats count,
sum(result_count) AS results, values(alert_actions) AS actions BY savedsearch_name
# The silent-source alert's search: each expected source's last event against its limit
| inputlookup sbd_expected_sources.csv | join type=left host [| tstats max(_time) AS last
where index=sbd_* earliest=-7d by host] | where isnull(last) OR last <
relative_time(now(), "-" . max_quiet_minutes . "m")
# The pipeline test, on SBD-SYSLOG01 (Tier 1 session): stop the forwarder past the silence
threshold
Stop-Service SplunkForwarder; Start-Sleep -Seconds <silence threshold plus one run
interval, in seconds>; Start-Service SplunkForwarder
```

> **WHAT SUCCESS LOOKS LIKE** Every alert in the register has a recorded threshold, schedule, expiration, severity, responder with deputy, and written first response; each safe condition fired its alert in a test, with the cause and the alert's time recorded; the forwarder stop fired the pipeline alert and its backlog arrived without a gap; and the first daily and weekly reviews are on record.

Your record

Evidence to keep: Keep the alert register, the listing of the alerts as built, each test's cause-and-fire record, the seven-day tuning counts with any changes, and the first review records.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	

RECORD	YOUR OBSERVATION
Evidence file names	
Open items or findings	

Think it through

What makes an alert real rather than decorative?

> **REMEMBER** Every alert has a recorded threshold, an owner, and a written first response. The pipeline's own failures and the repository's findings now raise alerts. Each alert was caused, tuned, and put on a review cadence.

Splunk · Define roles on the Splunk platform with capabilities

Splunk · Getting started with alerts

Splunk · Monitor triggered alerts

NIST · SP 800-53 Rev. 5 security controls (AU family)

LESSON 05 / ABOUT 30 MIN

Prove the chain and hand off

> **WHY THIS MATTERS** One deliberate event, followed all the way: a failed privileged sign-in should appear on the source, the collector, the analyst's screen, and the repository's next-morning archive. That trace closes all three logging courses; SBD-SPLUNK01's first baseline and the handoff to the fleet close this one.

In this lesson you will

- Trace a test event through all three systems, from source to analyst.
- Verify the router-deny stream (rule F24) and its alert on the analysis side.
- Baseline SBD-SPLUNK01 and file the evidence of all three logging courses.

Follow the build

1. Generate the test event: one failed sign-in on SBD-DC01 with a wrong password for a tier admin account. From AdminJumpBox01, record its time, account, and event ID in the DC's Security log, then find the same event in ForwardedEvents on SBD-SYSLOG01 with MachineName SBD-DC01 and the same time, about 30 seconds after it happened.
2. Watch analysis catch it: the event appears in sbd_wineventlog with host SBD-DC01 and the same time, and the privileged sign-in alert fires on its next run. The responder sees it in the Triggered Alerts list and records the first response as written — the alert is part of the trace, not an extra.
3. The next morning, after course 1.11's 01:00 run has archived the event's completed UTC day, open that day's ForwardedEvents archive on SBD-AUDIT01 in an auditor's or the custodian's session, confirm its catalog line shows matching hashes, and read the event with Get-WinEvent -Path. Four places, one event, and the same source time in each: that is the chain working.
4. Repeat with the boundary on the analysis side: probe one unregistered port from SBD-ADMIN-SRV01 as course 1.10 did, which already traced the deny to the collector over rule F24, and find it in search two with the boundary alert firing for a deny from VLAN 60. Attempt, deny, and alert now form one evidence set.
5. Run the cadence: take SBD-SPLUNK01's first SCAP baseline the way course 1.9 scans every Linux server. Over your Tier 1 SSH session, copy SCC for Linux and the distribution's STIG benchmark from C:\Staging\DISA\linux, check their hashes on the host, install SCC as its readme directs, and run it as root with the benchmark and the profile course 1.4 recorded. Answer DISA's Splunk Enterprise STIG for the application in a STIG Viewer checklist on SBD-ADMIN-SRV01, classify every open item as course 1.4 taught, adapted for Linux as in course 1.9 — fix now (no GPO applies to Linux), deviation candidate, or POA&M candidate — and store the session under C:\Evidence\splunk-analysis\baseline\SBD-SPLUNK01 with hashes.
6. File the end-to-end trace as the signature evidence of all three logging courses and write the handoff: the fleet course inherits a pipeline that will see its first domain join happen. When course 1.13 enrolls the workstations, their events arrive in sbd_wineventlog with no change in Splunk; add the ones that must never go quiet to the expected-sources lookup, and recheck the index sizing and the license's daily volume.

Check your result

```
# From AdminJumpBox01 (Tier 0): the test event on its source
```

```
Invoke-Command SBD-DC01 { Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4625;
StartTime=(Get-Date).AddHours(-1)} | Select-Object TimeCreated, Id, MachineName }
# On SBD-SYSLOG01 (Tier 1 session): the same event on the collector
Get-WinEvent -FilterHashtable @{LogName='ForwardedEvents'; Id=4625; StartTime=(Get-
Date).AddHours(-1)} | Select-Object TimeCreated, MachineName
# In Splunk Web (Search), as an analyst: the same event; its alert is in Triggered Alerts
index=sbd_wineventlog host=SBD-DC01* EventCode=4625 earliest=-1h | table _time host
TargetUserName
# Next morning, in an auditor's or the custodian's session on SBD-AUDIT01: that UTC day's
archive
Get-WinEvent -Path D:\LogArchive\ForwardedEvents\ForwardedEvents-<UTC date>.evtx -
FilterXPath '*[System[EventID=4625]]' | Select-Object TimeCreated, MachineName
# On SBD-ADMIN-SRV01: copy course 1.9's Linux assessment set to the host
scp C:\Staging\DISA\linux\<SCC for Linux package> C:\Staging\DISA\linux\<STIG benchmark>
<your-account>@SBD-SPLUNK01.sbd.test:/tmp/
# On SBD-SPLUNK01: the same hashes, then SCC's command-line scanner as root (path and
options from the SCC readme)
sha256sum /tmp/<SCC for Linux package> /tmp/<STIG benchmark>
sudo <SCC install directory>/cscc
# On SBD-ADMIN-SRV01: collect the session, then hash the course's evidence
New-Item -ItemType Directory -Force C:\Evidence\splunk-analysis\baseline | Out-Null
scp -r <your-account>@SBD-SPLUNK01.sbd.test:<SCC session folder> C:\Evidence\splunk-
analysis\baseline\SBD-SPLUNK01
Get-ChildItem C:\Evidence\splunk-analysis -Recurse -File | Get-FileHash -Algorithm SHA256
```

WHAT SUCCESS LOOKS LIKE The test event appears on SBD-DC01, in ForwardedEvents, in Splunk with its alert fired, and in the next morning's archive with matching catalog hashes, all with the same source time; the probe's deny appears in search two and fired its alert; and SBD-SPLUNK01 has its first baseline and Splunk STIG checklist with every open item classified.

Your record

Evidence to keep: Keep the four-place trace of the test event with its timestamps and the alert's record, the deny with its alert, SBD-SPLUNK01's scan set and checklist with hashes, and the handoff note to course 1.13.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

What does the single traced event prove that stage-by-stage checks don't?

REMEMBER One event, four places, one source time — the chain is real. Boundary denies assemble their own evidence, alert included. SBD-SPLUNK01's baseline and the handoff to course 1.13 close the logging courses.

Microsoft · Wecutil (Windows Event Collector)

Microsoft · Get-WinEvent

DISA · Security Content Automation Protocol (SCC and SCAP benchmarks)

DISA · STIGs document library (operating system and Splunk Enterprise STIGs)

DISA · SRG/STIG tools (SCC and STIG Viewer)

NIST · SP 800-53 Rev. 5 security controls (AU family)

Course review and sign-off

Before moving on, confirm each lesson left evidence behind — and that what remains open is written down, not remembered.

Lesson completion

LESSON	EVIDENCE FILED	DATE
01 Plan analysis and build SBD-SPLUNK01	☐ yes	
02 Install Splunk and bring in the streams	☐ yes	
03 Build the working searches	☐ yes	
04 Own the alerts	☐ yes	
05 Prove the chain and hand off	☐ yes	

Open items carried forward

Next in the learning path

That completes course 1.12 and the three logging courses. Course 1.13 brings in the fifty-workstation fleet — watched, addressed, and policy-managed from first boot.

> **KEEP IT HONEST** Lesson completion records learning activity; it is not a system authorization decision. Your observations, not the sample values, establish what happened in your environment.

© 2026 Secure By Design Team LLC.